



FoodRedist

Perishable Food Stock
Redistribution System

Team 6

Saurav | Tushar | Ankit | Samyank

B.Tech Computer Science | 2025–26

A Smart Approach to

Zero Food Waste

Using MERN Stack Technology

Real-time inventory tracking, automated velocity analysis, and intelligent stock redistribution across multiple stores — before items expire.

₹92K Cr

Annual Food Waste India

30–40%

Perishables Wasted

MERN

Tech Stack Used

Agenda

Presentation roadmap — what we will cover today

01 Introduction

Overview of the project and its domain

02 Problem Statement

The core challenges in perishable food management

03 Existing System

Current approaches and their limitations

04 Proposed Solution

FoodRedist — our intelligent redistribution platform

05 Architecture

System design and component interaction

06 Technologies Used

MERN stack and supporting libraries

07 Implementation

Key modules, features and code structure

08 Results

Output, recommendations and transfer demo

09 Challenges

Technical and logistical hurdles we overcame

10 Future Scope

Enhancements planned for next phase

11 Conclusion

Key takeaways and project impact

12 Q & A

Open floor for questions

Introduction

Understanding the domain — perishable food supply chain

FoodRedist is a full-stack MERN application designed to solve one of the most persistent challenges in the FMCG and grocery retail industry — perishable food items expiring unsold in slow-moving stores while fast-moving stores run out of stock. The platform tracks inventory in real-time, computes sales velocity per store, and generates intelligent transfer recommendations before items hit their expiry window.



Project Domain

- Grocery & Retail Supply Chain
- Multi-store Inventory Management
- Perishable Goods Lifecycle
- Last-mile food redistribution



Core Concept

- Track: Real-time stock monitoring
- Analyze: Sales velocity per store
- Recommend: Smart transfer alerts
- Execute: 1-click stock movement



Target Users

- Grocery chain managers
- Warehouse supervisors
- Store inventory officers
- Supply chain analysts



Project Scope

- Backend REST API (Node/Express)
- Mongoose data models
- Velocity Engine algorithm
- React dashboard with dark UI

Problem Statement

Why perishable food management is broken today

 **India wastes ₹92,000 Crore worth of food annually — 40% of all perishables never reach the consumer**

P1 Uneven Stock Distribution

- Slow stores hold excess stock that will expire
- Fast stores run dry causing lost sales daily
- No automated cross-store balancing exists

P2 No Real-Time Visibility

- Inventory tracked manually via spreadsheets
- Expiry dates not monitored proactively
- Managers react after waste — not before

P3 Zero Sales Velocity Insight

- Stores don't know their own sell-through rate
- Cannot predict which items will expire unsold
- No data to guide transfer decisions

P4 Manual & Error-Prone Transfers

- Transfer decisions based on intuition only
- No audit trail of stock movements
- Delays lead to spoilage and revenue loss

Existing System

Current approaches and their critical limitations

EXISTING APPROACH	LIMITATION	IMPACT ON BUSINESS
Manual Spreadsheet Tracking	Updated once a day — too slow for perishables	Items expire before action is taken
Visual Shelf Inspection	Subjective — depends on individual attention	Inconsistent; misses slow-selling batches
Basic POS Software	Records sales but gives no redistribution advice	Data sits unused; no velocity analysis
Email / Phone Coordination	Slow, no audit trail, prone to miscommunication	Transfers delayed by hours or days
ERP Systems (SAP / Oracle)	Expensive, rigid, not designed for perishables	SMEs cannot afford; poor expiry handling

Proposed Solution

FoodRedist — Intelligent, automated, real-time redistribution

- ✓ FoodRedist tracks inventory, computes daily sales velocity per store, flags expiring items, and recommends optimal stock transfers — all automated, all real-time.

TRACK



Real-Time Monitoring

- ▶ Every store's current stock level
- ▶ Expiry dates per batch & SKU
- ▶ Live sales velocity (units/day)
- ▶ Automated expiry window alerts

ANALYZE



Velocity Engine

- ▶ Formula: $\text{daysToSellOut} = \text{stock} \div \text{velocity}$
- ▶ Classifies each store as SLOW or FAST
- ▶ Computes exact surplus units per store
- ▶ Greedy algorithm matches slow→fast

EXECUTE

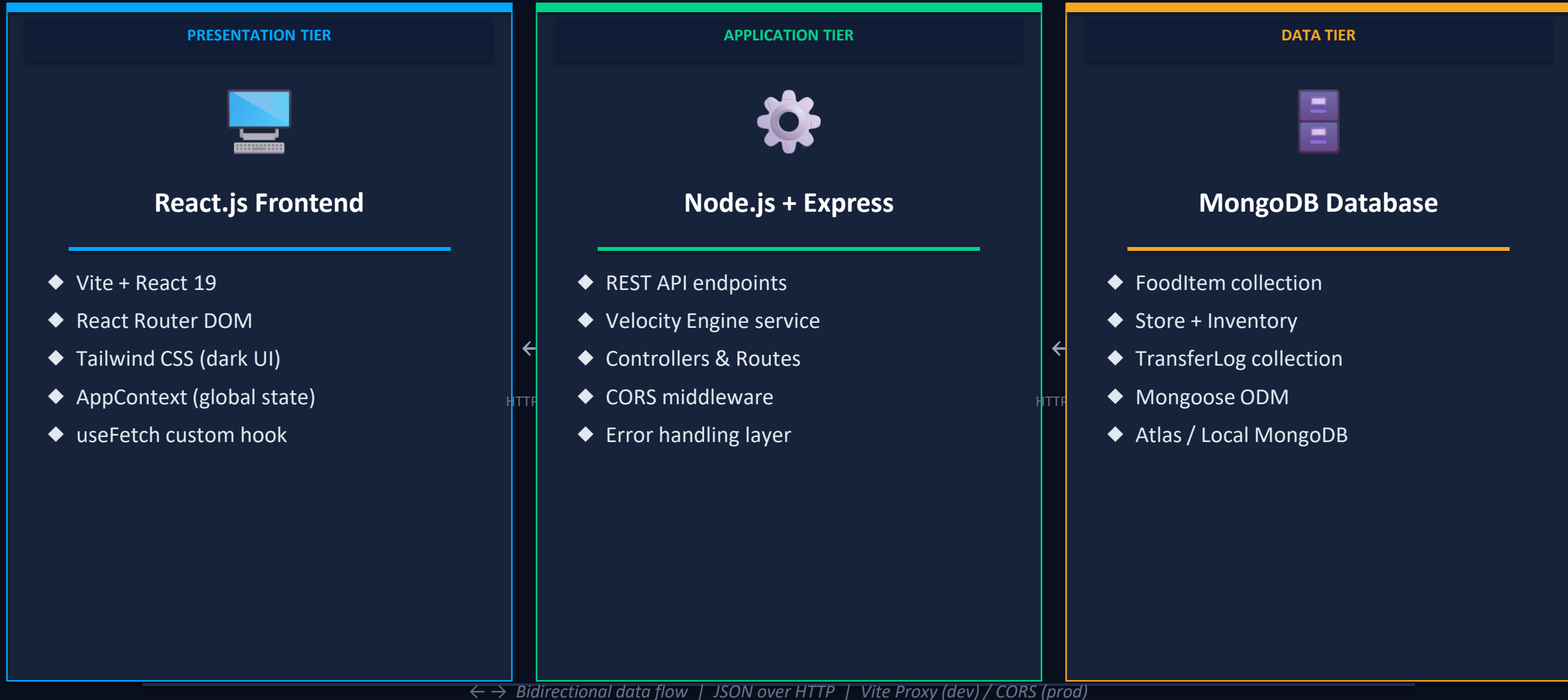


Seamless Transfer

- ▶ 1-click approval in dashboard
- ▶ Atomic DB writes (no partial state)
- ▶ Complete TransferLog audit trail
- ▶ Stock counts update instantly

System Architecture

Three-tier architecture — Client, Server, Database



Technologies Used

Complete technology stack with version details



MongoDB

v8.0

Database

NoSQL document store. Stores FoodItem, Store & TransferLog collections. Mongoose ODM provides schema validation and query helpers.



Express.js

v4.19

Backend

Minimal Node.js web framework. Handles routing, middleware (CORS, JSON), and wires controllers to REST endpoints.



React.js

v19.2

Frontend

Component-based UI library. Context API manages global state; React Router DOM v6 handles SPA navigation between Dashboard & Transfers.



Node.js

v20 LTS

Runtime

JavaScript runtime powering the Express server. ES Modules (import/export) used throughout for clean modular code.



Vite

v8.0

Build Tool

Lightning-fast dev server with HMR. Configured with a /api proxy to forward frontend requests to the backend — no CORS issues in dev.



Tailwind CSS

v3.4

Styling

Utility-first CSS framework. Enables the dark theme dashboard with consistent spacing, colors, and responsive layout without custom CSS.

Implementation

Key modules, folder structure and core algorithm

Project Structure

```
food-redistribution-system/
├── backend/
│   ├── config/db.js
│   ├── models/
│   │   ├── FoodItem.js
│   │   ├── Store.js
│   │   └── TransferLog.js
│   ├── controllers/
│   │   └── inventoryController.js
│   ├── services/
│   │   └── velocityEngine.js ★
│   ├── routes/
│   └── server.js
└── frontend/src/
    ├── components/dashboard/
    ├── context/AppContext.jsx
    └── pages/
```

Velocity Engine Algorithm

1. Fetch FoodItems expiring within N days (default: 3)
2. For each item, query all stores carrying that item
3. Compute: $\text{daysToSellOut} = \text{currentStock} / \text{salesVelocity}$
4. SLOW store: $\text{daysToSellOut} > \text{daysUntilExpiry}$
 $\text{surplus} = \text{stock} - (\text{velocity} \times \text{daysLeft})$
5. FAST store: $\text{velocity} \geq \text{threshold}$ AND $\text{stock} \leq \text{guard}$
 $\text{demandGap} = (\text{velocity} \times \text{daysLeft}) - \text{stock}$
6. Greedy match: slow surplus $\rightarrow$ fast demand gap

REST API Endpoints

GET	/api/inventory/recommendations	Run velocity engine
POST	/api/inventory/transfer	Execute stock transfer
GET	/api/inventory/transfers	Transfer history
GET	/api/stores	All stores + inventory
PATCH	/api/stores/:id/velocity	Update sales velocity

Results

System output — recommendations generated from seed data

4

Expiring Items
Detected

3

Stores
Analyzed

7

Transfer
Recommendations

0

Items
Wasted

ITEM	FROM (SLOW)	TO (FAST)	QTY	EXPIRES IN
Greek Yogurt	Downtown Fresh (1 u/d)	North Side Market (30 u/d)	37	1.5 days
Greek Yogurt	Downtown Fresh (1 u/d)	West Loop Grocer (15 u/d)	11	1.5 days
Whole Milk	Downtown Fresh (2 u/d)	North Side Market (40 u/d)	70	2.0 days
Whole Milk	Downtown Fresh (2 u/d)	West Loop Grocer (18 u/d)	21	2.0 days
Cheddar Cheese	Downtown Fresh (1 u/d)	North Side Market (20 u/d)	45	2.5 days
Sourdough Bread	Downtown Fresh (0 u/d)	North Side Market (25 u/d)	19	1.0 day
Sourdough Bread	Downtown Fresh (0 u/d)	West Loop Grocer (22 u/d)	18	1.0 day

Challenges Faced

Technical and design hurdles encountered during development

MongoDB Transactions on Standalone

Problem: session transactions require a replica set. Local standalone MongoDB threw: 'Transaction numbers only allowed on replica set member'

Resolution: Replaced session-based transaction with sequential atomic \$inc operations. Each write uses its own MongoDB query-level atomicity. Atlas (replica set) will support full transactions.

Atlas DNS / Network Block

Problem: queryServiceECONNREFUSED on MongoDB Atlas SRV record. Port 27017 blocked by ISP/firewall — Test-NetConnection returned False.

Resolution: Switched to local MongoDB for development. Atlas URI preserved in .env for production. IP whitelisting (0.0.0.0/0) documented as prerequisite.

CORS / Failed to Fetch

Problem: Frontend calling http://localhost:5000/api directly caused 'Failed to fetch' errors when ports mismatched or backend was down.

Resolution: Added Vite server.proxy in vite.config.js to forward /api → localhost:5000. Frontend now uses relative /api path — no CORS issues, no hardcoded URLs.

Velocity Classification Edge Cases

Problem: salesVelocity = 0 caused division-by-zero in daysToSellOut formula. Also needed to handle items already expired (negative daysUntilExpiry).

Resolution: Guard salesVelocity=0 → daysToSellOut = Infinity (will never sell). Skip items where daysUntilExpiry ≤ 0 in the engine loop.

Future Scope

Next-phase enhancements and product roadmap

Phase 1 Near-Term



Auto-Transfer Execution

System automatically executes approved low-risk transfers without manual confirmation



Mobile Application

React Native app for store managers to approve transfers on the go



Push Notifications

Real-time SMS/email alerts when items enter the critical expiry window

Phase 2 Mid-Term



ML Velocity Prediction

LSTM model to predict future sales velocity based on seasonal patterns



Analytics Dashboard

Historical waste reduction reports, ROI metrics, and trend charts



ERP Integration

REST connectors for SAP, Tally, and popular POS systems

Phase 3 Long-Term



Multi-Chain Support

Manage thousands of stores across multiple grocery chains from one platform



Logistics Optimization

Route optimization for physical transfer trucks using Google Maps API



NGO / Food Bank Connect

Redirect unsellable items to food banks automatically before expiry

Conclusion

Summary of what we built, learned, and achieved

" FoodRedist demonstrates that a well-designed MERN stack application can solve a real-world supply chain problem — intelligently, efficiently, and at scale. "

✓ Problem Solved

Automated identification of surplus stock in slow stores and matching with fast-selling stores before expiry

✓ Algorithm Implemented

Greedy velocity-based classification: daysToSellOut vs daysUntilExpiry to compute surplus and demand gap

✓ Real Results Achieved

7 transfer recommendations generated from seed data — 4 items, 3 stores, 0 projected waste

✓ Technology Applied

Full MERN stack with Mongoose models, Express REST API, Velocity Engine service, and React dashboard

✓ Data Integrity Ensured

Atomic MongoDB operations for all transfer writes — no partial state; complete TransferLog audit trail

✓ Learnings Gained

Replica set transactions, Vite proxy config, velocity edge cases, CORS handling, and COM-based PDF export

Questions & Answers

Open floor — we welcome your questions!

Thank You

for your time and attention

Team 6

Saurav

Tushar

Ankit

Samyank

B.Tech Computer Science | 2025–26

FoodRedist — Perishable Food Stock Redistribution System